

- 1) Create la classe **IntegerSet** (insieme di interi). Ogni oggetto di **IntegerSet** può memorizzare interi fra 0 e 99. Un insieme è rappresentato internamente da un array di booleani. L'elemento dell'array **a[i]** è **true** se l'intero **i** è contenuto nell'insieme. L'elemento dell'array **a[j]** è **false** se l'intero **j** non è contenuto nell'insieme. Il costruttore di default inizializza un insieme all'insieme vuoto, il cui array contiene soltanto **false**.
Scrivete i metodi che effettuano le comuni operazioni sugli insiemi. Per esempio, scrivete la funzione **unionOfIntegerSet** che crea un terzo insieme che è l'unione di due insiemi esistenti: un elemento del terzo insieme è **true** se è presente in almeno uno dei due insiemi origine. Scrivete il metodo **intersectionOfIntegerSet** che crea un terzo insieme che è l'intersezione di due insiemi esistenti: un elemento del terzo insieme è **true** se è presente in entrambi gli insiemi origine. Scrivete la funzione membro **insertElement** che inserisce un nuovo intero **k** nell'insieme (impostando **a[k]** a **true**). Scrivete la funzione membro **deleteElement** che elimina dall'insieme l'intero **m** (impostando **a[m]** a **false**). Scrivete la funzione membro **toString()** che restituisce una stringa che rappresenta un insieme come lista di numeri separati da spazi. Visualizzate – per l'insieme vuoto. Scrivete un programma di prova che faccia uso della classe **IntegerSet**. Istanziare diversi oggetti **IntegerSet**. Verificate la correttezza dei metodi.
- 2) Si scriva una nuova versione della classe **IntegerSet** che ha gli stessi metodi della classe precedente ma rimuove il limite che gli interi devono essere compresi fra 0 e 99. Utilizzate una `ArrayList<Integer>` per memorizzare gli elementi dell'insieme e il metodo `get` o altri metodi che possono tornare utili che trovate già pronti in `ArrayList`.
Verificate che il main dell'esercizio 3 funziona anche con la nuova classe **IntegerSet** senza necessità di alcuna modifica (grazie all'incapsulamento).

ESERCIZI DEL LIBRO DI TESTO

Esercizio 4.26 Ordinamento di ArrayList con Comparator

Considerare le stringhe "luce", "la", "Urano", "su" e "Arriva", e inserirle in quest'ordine in un `ArrayList`. Stampare l'oggetto `arraylist` per verificare che l'ordine delle stringhe corrisponda all'ordine di inserimento. Creare una classe che implementa `Comparator` che implementa un criterio di ordinamento alfabetico (ignorando se le lettere sono minuscole o maiuscole). Ordinare mediante il metodo `sort` di `Collections`. L'output finale deve essere
[Arriva, la, luce, su, Urano]

Esercizio 5.4 Semplice algoritmo

Creare un file con un metodo `main` che con un ciclo `while` e una condizione `if`, riempie un `arraylist` di interi con i valori 1, 2, 3, 4, 6, 8, 10. Infine, stampare l'`arraylist`.

Esercizio 5.7 Tavola Pitagorica

Creare un file con un metodo `main` che stampi a video la tavola pitagorica.

Suggerimento 1: non sono necessari array.

Suggerimento 2: usare il metodo `System.out.print` (che non va a capo) per stampare su una riga. Usare `System.out.println()` senza parametri per andare a capo.

Suggerimento 3: sfruttare un doppio ciclo innestato.

Suggerimento 4: usare il carattere di escape `"\t"` (cfr. paragrafo 3.3.4) per distanziare i numeri in maniera uniforme su una riga.

Esercizio 5.14 Numeri pari con Iterator

Creare un file con un metodo `main` che, con un ciclo `while` e una condizione `if`, elimini i numeri dispari da un `arraylist` contenente i primi 10 numeri interi. Infine, stampare l'`arraylist`.

Esercizio 5.15 Inserimento di un numero positivo

Creare un programma che utilizza un ciclo `do-while` per chiedere all'utente di inserire un numero positivo (usare la classe `Scanner` e il metodo `nextInt`). Se l'utente inserisce un numero negativo, il programma deve richiedere un altro numero. Il programma termina solo quando l'utente inserisce un numero positivo.

Esercizio 5.20

Spiegare a cosa serve la parola chiave `yield`.

Esercizio 5.23 Moderatore di parole

Si consideri il seguente programma interattivo:

```
import java.util.Scanner;
public class ProgrammaInterattivo {
    public static void main(String args[]) {
        Scanner scanner = new Scanner(System.in);
        String stringa = "";
        System.out.println("Digita qualcosa e batti il tasto "
            + "\"\\n\\n\"", oppure scrivi "\\n\\n\"fine\\n\" per terminare "
            + "il programma");
        // Assegniamo a stringa l'input utente e testiamo se vale "fine"
        while(!(stringa = scanner.next()).equals("fine")) {
            System.out.println("Hai digitato " + stringa.toUpperCase() + "!");
        }
        System.out.println("Fine programma!");
    }
}
```

Questa classe legge l'input da tastiera mediante la classe Scanner. Il metodo next usato nel costrutto while (con una sintassi complessa che comprende anche l'assegnazione alla variabile stringa) è un metodo bloccante (ovvero blocca l'esecuzione del codice in attesa di input da parte dell'utente) che legge l'input da tastiera sino a quando si preme il tasto enter (invio). Il programma termina quando si digita la parola "fine". Modificare il programma in modo tale che moderi le parole: ogni parola, prima di essere stampata, deve essere controllata, e nel caso coincida con una parola vietata, deve essere censurata. Le parole vietate sono memorizzate in una lista che creerete nel main. Suggerimento: usare contains per verificare se una parola è presente nella lista.

Esercizio 5.24 Gestione Turni

Consideriamo l'enumerazione DayOfWeek del package java.time, che definisce come elementi i giorni della settimana MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, e SUNDAY (cfr. documentazione standard). Creare una classe chiamata GestioneTurni che definisca un metodo dammiGiornoDellaSettimana, che restituisca in maniera casuale uno degli elementi di DayOfWeek. utilizzando un'espressione switch. Creare una classe TestGestioneTurni che istanzi la classe GestioneTurni, e stampi il risultato della chiamata al metodo dammiGiornoDellaSettimana.

Esercizio 5.25 Statistiche

Partendo dalla soluzione dell'esercizio 5.24, creare una classe StatisticheTurni con un metodo stampaStatistiche che prenda come parametro un numero intero numeroTurni. Il metodo deve ciclare in base al parametro numeroTurni, per chiamare il metodo dammiGiornoDellaSettimana della classe GestioneTurni, e incrementare dei contatori. Prendendo spunto dall'esempio riportato nel sottoparagrafo intitolato "Fall through" del paragrafo 5.3.1, incrementeremo la variabile numeroInizioSettimana se il giorno generato è MONDAY, la variabile numeroCentroSettimana se il giorno generato è TUESDAY, WEDNESDAY o THURSDAY, la variabile numeroFineSettimana se il giorno generato è FRIDAY o la variabile numeroWeekend se il giorno generato è SATURDAY o SUNDAY. Dopo il ciclo, stampare le statistiche delle quattro variabili. Infine, creare una classe TestStatisticheTurni che chiami il metodo stampaStatistiche della classe StatisticheTurni.

Esercizio 5.26 Statistiche con HashMap

Partendo dalla soluzione precedente, modificare la classe StatisticheTurni sostituendo alle variabili contatore una HashMap che associ ogni giorno della settimana al suo contatore. Modificare il metodo stampaStatistiche in modo tale che vengano stampate i contatori per singoli giorni. Notare che potete raggiungere l'obiettivo senza usare costrutti condizionali grazie alla proprietà della HashMap.

Progetto 5.2 JTombola

Creare un progetto che simula il gioco della Tombola. Come primo passo, creare una classe che si chiama Bussolotto che contiene un ArrayList contenente i 90 numeri da estrarre, e un metodo dammiNumero che restituisce un numero casuale da 1 a 90. Creare anche una classe di test chiamata TestBussolotto che chiama 90 volte il metodo dammiNumero della classe Bussolotto. Cosa succede se il metodo viene chiamato più di 90 volte?